

Q34. Case Study: Robotics Vision System (Matrix Multiplication)

1. Aim

To analyze the performance of matrix multiplication in a robotics vision system and apply the **Divide and Conquer approach using Strassen's Matrix Multiplication** to improve computational efficiency.

2. Problem Statement

A robotics vision system uses matrix transformations to process images and visual information. Traditional matrix multiplication requires a large number of multiplication operations, which can increase processing time for large matrices. Therefore, an optimized matrix multiplication technique is required for real-time robotic applications.

3. Objectives

- Analyze matrix multiplication in a robotics vision system.
- Identify computational challenges in real-time processing.
- Apply Divide and Conquer using Strassen's method.
- Compare the computational complexity of matrix multiplication methods.
- Justify the use of an optimized algorithm for large matrices.

4. Computational Challenges

The major challenges in real-time robotic vision processing are:

- Large image matrices require many calculations.
- Traditional multiplication becomes slower as matrix size increases.
- Real-time systems require fast processing.
- Memory usage increases for large matrices.
- Repeated matrix transformations can affect system response time.

5. Algorithm Used

Strassen's Matrix Multiplication using Divide and Conquer

Strassen's method divides a large matrix into smaller submatrices and performs multiplication recursively. It reduces the number of recursive matrix multiplications from **8 to 7**, improving performance for sufficiently large matrices.

6. Algorithm

1. Start with two square matrices.
2. Divide each matrix into four smaller submatrices.
3. Calculate seven intermediate products using Strassen's formulas.
4. Combine the intermediate results to obtain the four parts of the final matrix.
5. Continue recursively until the matrix size becomes small.
6. Return the final multiplied matrix.
7. Measure the execution time for different matrix sizes.

PROGRAM:

```
1 import numpy as np
2 import time
3
4 def strassen(A, B):
5     n = len(A)
6
7     if n == 1:
8         return A * B
9
10    mid = n // 2
11
12    A11 = A[:mid, :mid]
13    A12 = A[:mid, mid:]
14    A21 = A[mid:, :mid]
15    A22 = A[mid:, mid:]
16
17    B11 = B[:mid, :mid]
18    B12 = B[:mid, mid:]
19    B21 = B[mid:, :mid]
20    B22 = B[mid:, mid:]
21
22    M1 = strassen(A11 + A22, B11 + B22)
23    M2 = strassen(A21 + A22, B11)
24    M3 = strassen(A11, B12 - B22)
25    M4 = strassen(A22, B21 - B11)
```

\

OUTPUT

```
Strassen Matrix Multiplication Performance
-----
Matrix Size 2x2 : 0.000071 seconds
Matrix Size 4x4 : 0.000265 seconds
Matrix Size 8x8 : 0.001569 seconds
Matrix Size 16x16 : 0.010596 seconds

=== Code Execution Successful ===
```

9. Analysis

As the matrix size increases, the execution time also increases because more elements have to be processed. Strassen's method reduces the number of multiplication operations by dividing the matrices into smaller parts and using seven recursive multiplications instead of eight.

For small matrices, the additional recursion and matrix additions may reduce the practical advantage. However, for larger matrices, the reduction in multiplication operations can improve performance.

10. Complexity Analysis

Traditional Matrix Multiplication

Time Complexity: $O(n^3)$

Strassen's Matrix Multiplication

Time Complexity: $O(n^{\log_2 7})$

$$\approx O(n^{2.807})$$

Therefore, Strassen's method has a lower asymptotic time complexity than traditional matrix multiplication.

11. Real-Time Processing Interpretation

In a robotics vision system, matrices may represent image transformations, object positions, rotations, or other visual information. Faster matrix multiplication can help the robot process these transformations more efficiently and respond quickly to changes in its environment.

12. Justification of the Solution

Strassen's algorithm is suitable for large matrix operations because:

- It uses Divide and Conquer.

- It reduces the number of multiplication operations.
- It has lower theoretical complexity than the standard method.
- It can improve performance for large matrices.
- It is useful when repeated matrix transformations are required.

For very small matrices, conventional multiplication may still be faster because Strassen's recursion introduces additional overhead.

13. Result

The Divide and Conquer approach using **Strassen's Matrix Multiplication** successfully reduces the theoretical computational complexity from $O(n^3)$ to approximately $O(n^{2.807})$. The method is therefore more suitable for large matrix operations in robotics vision systems.

14. Conclusion

Strassen's Matrix Multiplication provides an optimized approach for processing large matrices in robotics vision applications. By dividing matrices into smaller submatrices and reducing the number of multiplication operations, it can improve computational efficiency. Therefore, Strassen's method is a suitable solution when large matrix transformations must be performed efficiently.